

TLM: Deep Learning

Chapter 1: Neural Network Foundations with TF

Outline:

- (1) Deep learning libraries
- (2) Definisi ANN and Deep learning
- (3) Perceptron calculation
- (4) Multi Layer Perceptron (MLP) calculation
- (5) Problem in Training perceptron
- (6) The activation function in neural network
- (7) Neural network definition
- (8) A real example: recognizing handwritten digits
- (9) Defining a simple neural net in TensorFlow
- (10) Running a simple TensorFlow net and establishing a baseline and some other scenarios for improvement

Dosen Penyusun:

David Agustriawan, S.Kom., M.S.c., Ph.D.

Ajie Kusuma Wardhana, S.Kom., M.Eng

Deep learning libraries:

1. TensorFlow (TF)
2. PyTorch
3. Caffe
4. MXNet
5. Keras

Keras – a high level neural network API that has been integrated with TensorFlow (in 2.0 Keras became the standard API for interacting with TensorFlow). This API specifies how software components should interact. As of May 2022, TensorFlow is the most popular framework for machine learning.

Artificial neural networks (ANN)

Represents a class of machine learning models loosely inspired by studies about the central nervous systems of mammals. Each ANN is made up of several interconnected “neurons,” organized in “layers.” Neurons in one layers pass messages to the neurons in the next layer and this is how the network compute things.

Deep learning:

a class of neural networks characterized by a significant number of layers of neurons. People began referring to it as “deep” when it started utilized 3-5 layers. Now networks with more than 200 layers are commonplace.

Perceptron:

Perceptron was the name given to the model having one single linier layer.

The perceptron is a simple algorithm that, given an input vector x of m values ($x_1, x_2, \dots, x_m$), often called input features or simply features, output either a 1 (“yes”) or a 0 (“no”).

Mathematically, we define a function:

$$f(x) = \begin{cases} 1 & wx + b > 0 \\ 0 & \text{otherwise} \end{cases}$$

Where w is a vector of weights, $w \cdot x$ is the dot product $\sum_{j=1}^m w_j x_j$, and b is the bias. If you remember elementary geometry, $wx + b$ defines a boundary hyperplane that changes position according to the values assigned to w and b . Note that a hyperplane is a subspace whose dimension is one fewer than that of its ambient space. See (Figure 1.2) for an example:

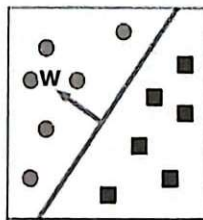
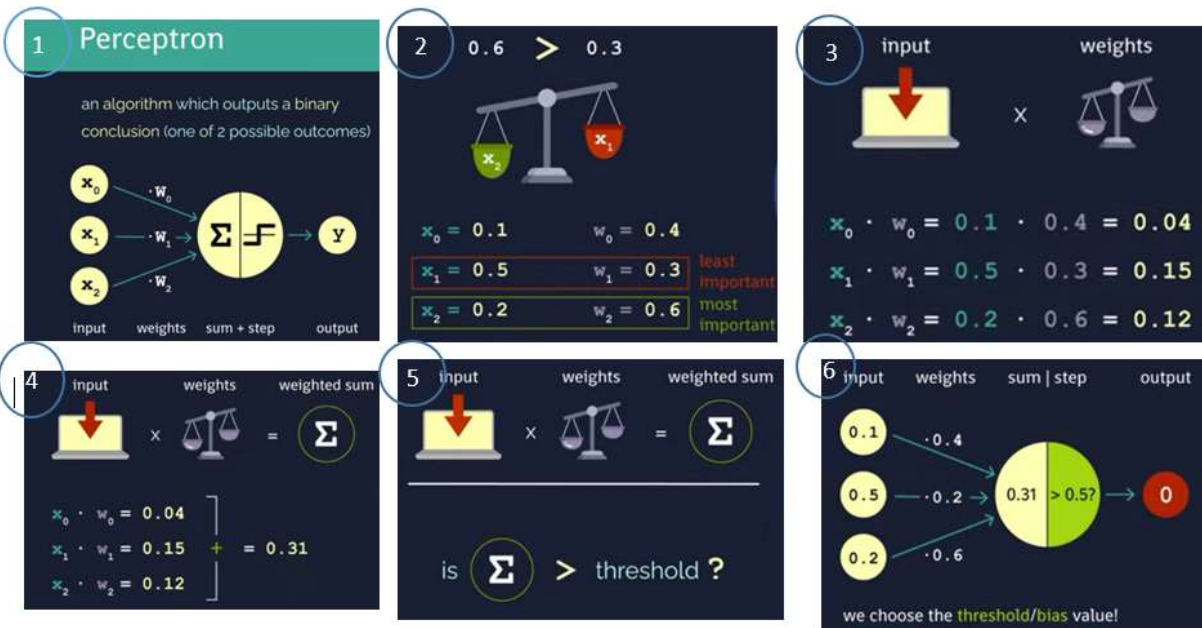


Figure 1.2: An example of a hyperplane



The accuracy will depend on how well we can adjust the weights and Bias/Threshold (Training process will do!!!!)

There are three ways of creating a model in tf.keras:

(1) sequential API, (2) functional API, and (3) model subclassing.

A **Sequential()** model is a linier pipeline (a stack) of neural network layers.

Below is the example of the simplest one, Sequential():

```
import tensorflow as tf

from tensorflow import keras

NB_CLASSES = 10

RESHAPE = 784

model = tf.keras.models.Sequential()

model.add(keras.layers.Dense(NB_CLASSES,
                              input_shape=(RESHAPED,), kernel_initializer='zeros',
                              name='dense_layer', activation='softmax'))
```

This code fragment **defines a single layer** with **10 artificial neurons** that **expect 784 input variables (also known as features)**. Note that the net (network) is “dense”, meaning that each neuron in a layer is connected to all neurons located in the previous layer, and to all the neurons in the following layer.

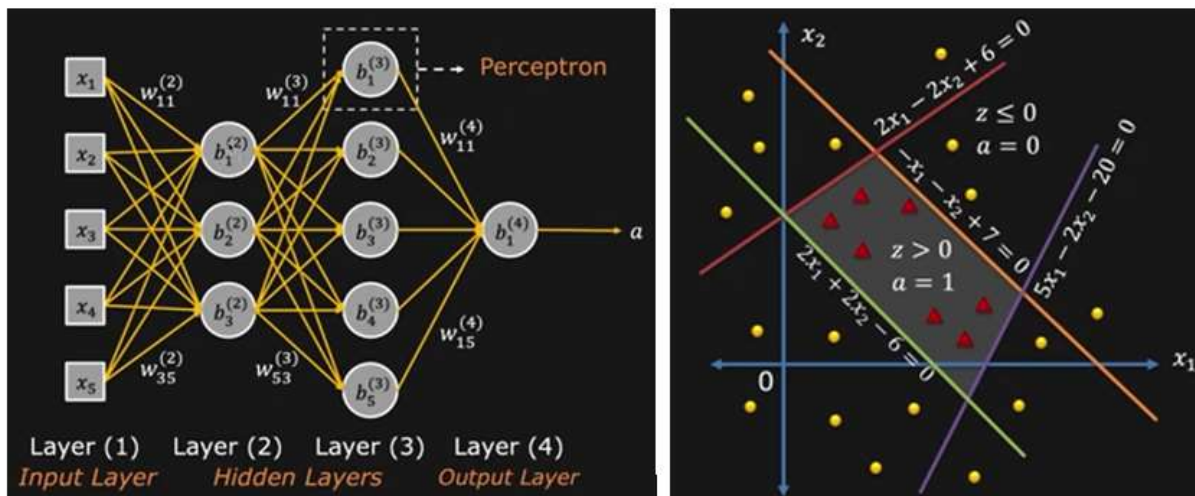
Each neuron can be **initialized with specific weights via the 'kernel_initializer' parameter**. There are a few choices, the most common of which are listed below:

- **random_uniform**: weights are initialized to uniformly random small values in the range (-0.05, 0.05)
- **random_normal**: weights are initialized according to a Gaussian distribution with zero mean and small standard deviation of 0.05
- **zero**: all weights are initialized to zero.

Multi-Layer Perceptron (MLP)

MLP is a network with multiple dense layers. **Note that the input and output layers are visible from the outside, while all the others layers in the middle are hidden – hence the name hidden layer.**

Contoh arsitektur MLP: 5 datasets, 2 hidden layer (hidden layer 1 = 3 neurons, hidden layer 2 = 5 neurons dan 1 output layer. Dan dataset nonlinear.



Contoh perhitungan pada MLP:

Skenario Data

- **Input (X):** $[0.5, 0.1, -0.2]$ (3 fitur)
- **Target (y):** 1 (Binary)
- **Arsitektur:** 3 Input $\rightarrow$ 2 Hidden Neurons $\rightarrow$ 1 Output Neuron.
- **Fungsi Aktivasi:** Sigmoid $\sigma(x) = \frac{1}{1+e^{-x}}$.

Note: 3 input maksudnya 3 features yang di flattened yang akan dihitung pada setiap neuron pada hidden layers.

Setiap neuron di hidden layer punya bobot (W) dan bias (b) masing-masing. Misalkan bobot awalnya adalah:

- **Neuron h_1 :** $W_{h1} = [0.1, 0.3, -0.1]$, $b_1 = 0.2$
- **Neuron h_2 :** $W_{h2} = [-0.2, 0.4, 0.5]$, $b_2 = -0.1$

Note: bobotnya ada 3 pada setiap neuron di hidden layer dikarenakan terdapat 3 features pada dataset input

Hitung Skor Linear (z):

- $z_{h1} = (0.5 \times 0.1) + (0.1 \times 0.3) + (-0.2 \times -0.1) + 0.2 = 0.05 + 0.03 + 0.02 + 0.2 = \mathbf{0.3}$
- $z_{h2} = (0.5 \times -0.2) + (0.1 \times 0.4) + (-0.2 \times 0.5) - 0.1 = -0.1 + 0.04 - 0.1 - 0.1 = \mathbf{-0.26}$

Note: skor linear adalah dot product antara weight dan nilai input ditambah bias.

Hitung Aktivasi Sigmoid (a):

- $a_{h1} = \sigma(0.3) = \frac{1}{1+e^{-0.3}} \approx \mathbf{0.574}$
- $a_{h2} = \sigma(-0.26) = \frac{1}{1+e^{0.26}} \approx \mathbf{0.435}$

Sekarang, hasil dari hidden layer (a_{h1} , a_{h2}) menjadi input untuk neuron terakhir.

Misalkan bobot outputnya: $W_{out} = [0.7, -0.5]$ dan $b_{out} = 0.1$.

Hitung Skor Linear Output (z_{out}):

$$z_{out} = (a_{h1} \times w_{out1}) + (a_{h2} \times w_{out2}) + b_{out}$$

$$z_{out} = (0.574 \times 0.7) + (0.435 \times -0.5) + 0.1$$

$$z_{out} = 0.4018 - 0.2175 + 0.1 = \mathbf{0.2843}$$

Hitung Prediksi Akhir ($\hat{y}$):

$$\hat{y} = \sigma(0.2843) = \frac{1}{1+e^{-0.2843}} \approx \mathbf{0.570}$$

Hasil prediksi kita adalah **0.570**. Karena ini klasifikasi binary:

- Jika menggunakan ambang batas (*threshold*) **0.5**, maka model menebak **Kelas 1**.
- **Error:** Karena target asli $y = 1$, maka errornya kecil ($1 - 0.57 = 0.43$).

Jika hasil di atas salah (misal $\hat{y}$ jauh dari target), model akan menghitung **Loss** (misalnya menggunakan *Mean Squared Error* atau *Log Loss*).

Secara matematis, bobot diupdate dengan rumus:

$$W_{new} = W_{old} - \eta \frac{\partial Loss}{\partial W}$$

Artinya: "Bobot baru adalah bobot lama dikurangi (Laju Pembelajaran $\times$ Turunan Error terhadap Bobot tersebut)".

Bagaimana mendapatkan bobot optimal untuk 1000 dataset?

1. **Inisialisasi:** Model mulai dengan bobot acak (ngawur).
2. **Iterasi (Epoch):** * Model mencoba memprediksi data ke-1 sampai ke-1.000.
 - Setiap kali model salah (misal pada data ke-50), model akan **mengubah sedikit** bobotnya agar lebih akurat untuk data ke-50 tersebut.
 - Namun, perubahan ini juga harus tetap membuat model akurat untuk data ke-1 sampai ke-49.
3. **Konvergensi:** Setelah berulang kali melihat 1.000 data tersebut (ratusan kali putaran/epoch), bobot akan berhenti berubah. Bobot inilah yang kita sebut **Bobot Optimal**.

Problems in training the perceptron and solution

Let's consider a single neuron; what are the best choices for the weight w and bias b ?

Ideally, we would like to provide a set of training examples and let the computer adjust the weight and the bias in such a way that the errors produced in the output are minimized. This approach seems very intuitive, but it requires that a small change in the weights (or the bias) causes only small change in the outputs.

if we have a big output jump, we cannot learn progressively. Unfortunately, the perceptron does not show this "little by little" behavior. A perceptron is either a 0 or 1, and that's a big jump that will not help in learning.

We need function that progressively changes from 0 to 1 with no discontinuity. Mathematically, this means that we need continuous function that allows us to compute the derivative. In mathematics the derivative is the amount by which a function is changing at a given point.

Table Activation function in neural network

Activation function	Explanation
sigmoid	range (0,1). Note, a neuron with sigmoid activation has a behavior similar to the perceptron, but the changes are gradual
tanh	range (-1,1)
ReLU	a ReLU is simply defined as $f(x) = \max(0, x)$. The function is zero for negative values and it grows linearly for positive values
Etc	

In short: what are the neural networks after all?

Machine learning models are a way to compute a function that maps some inputs to their corresponding outputs. The function is nothing more than a number of addition and multiplication operations. However, when combined with a nonlinear activation and stacked in multiple layers, these functions can learn almost anything.

A real example: recognizing handwritten digits.

Labelled dataset: MNIST (<https://www.kaggle.com/datasets/hojjatk/mnist-dataset>). Each MNIST image is in grayscale and consists of 28 x 28 pixels.

One hot-encoding (OHE)

OHE is used as a simple tool to encode information used inside neural networks. In many applications, it is convenient to transform categorical (non-numerical) features into numerical variables. For instance, the categorical feature digit with value d in $[0-9]$ can be encoded into a binary vector with 10 positions, which has always a 0 value except the d -th position where a 1 is present. For example, the digit 3 can be encoded as $[0,0,0,1,0,0,0,0,0,0]$.

One-Hot Encoding mengubah label menjadi vektor biner di mana hanya ada satu angka **1** (hot) dan sisanya **0** (cold).

- Angka 1 → $[1, 0, 0, 0, 0, 0, 0, 0, 0, 0]$
- Angka 2 → $[0, 1, 0, 0, 0, 0, 0, 0, 0, 0]$
- Angka 9 → $[0, 0, 0, 0, 0, 0, 0, 0, 0, 1]$

Dengan format ini, jarak antara setiap kategori adalah **sama**. Model tidak akan menganggap angka 1 lebih mirip dengan 2 dibandingkan dengan 9. Semuanya dianggap sebagai entitas yang unik dan independen.

Jika Anda menggunakan label numerik (1, 2, 3, . . . , 9), model secara matematis akan menganggap bahwa angka-angka tersebut memiliki **urutan dan jarak**.

- **Jarak Matematika:** Model akan berpikir bahwa angka **1** lebih dekat ke angka **2** daripada ke angka **9**.
- **Implikasi Salah:** Dalam klasifikasi gambar, gambar angka "1" sama sekali tidak ada hubungannya dengan angka "2". Mereka adalah kategori yang berbeda secara total.
- **Error Bias:** Jika model menebak angka **1** padahal seharusnya **9**, nilai error-nya akan sangat besar ($|9 - 1| = 8$). Namun jika menebak **8** padahal seharusnya **9**, error-nya kecil ($|9 - 8| = 1$). Ini menyesatkan model karena dalam klasifikasi, **salah tetaplah salah**, tidak peduli seberapa dekat angka tersebut secara urutan.

Defining a simple neural net in TensorFlow

TensorFlow provides suitable libraries (https://www.tensorflow.org/api_docs/python/tf/keras/datasets) for loading the dataset and splits it into training sets, X_train, used for fine tuning our net, and test sets, X_test, used for assessing the performance.

Data is converted into float32 to use 32-bit precision when training a neural network and normalized to the range [0,1]. Then, load the true labels into Y_train and Y_test respectively, and perform one-hot encoding on the dataset.

Epoch defines how long the training should last. **BATCH_SIZE** is the number of samples you feed in your network at a time, and the validation sample is the amount of data reserved for checking or proving the validity of training process

```
import tensorflow as tf
import numpy as np
from tensorflow import keras

#network and training parameters.
EPOCHS = 200
BATCH_SIZE = 128
VERBOSE = 1
NB_CLASSES = 10 # number of outputs = number of digits
N_HIDDEN = 128
VALIDATION_SPLIT = 0.2 # how much train is reserved for validation

#Loading MNIST dataset
#verify
#you can verify that the split between train and test is 60,000 and 10,000 respectively.
#labels have one-hot representation is automatically applied

mnist = keras.datasets.mnist
(X_train, Y_train), (X_test, Y_test)=mnist.load_data()
#X_train is 6000 rows of 28x28 values; we → reshape it to 6000 x784
RESHAPE = 784
#
X_train = X_train.reshape(60000, RESHAPE)
X_test = X_test.reshape(10000, RESHAPE)
X_train = X_train.astype('float32')
X_test = X_test.astype('float32')
```

#normalize inputs to be within in [0,1].

```
X_train /= 255
```

```
X_test /= 255
```

```
print(X_train.shape[0], 'train samples')
```

```
print(X_test.shape[0], 'test samples')
```

#One-hot representation of the labels

```
Y_train = tf.keras.utils.to_categorical(Y_train, NB_CLASSES)
```

```
Y_test = tf.keras.utils.to_categorical(Y_test, NB_CLASSES)
```

From the above code, input layer has a neuron associated to each pixel in the image for a total of $28 \times 28 = 784$ neurons, one for each pixel in the MNIST images. Typically, the values associated with each pixel are normalized in the range $[0,1]$ (which means that the intensity of each pixel is divided by 255, the maximum intensity value). The output can be one of ten classes, with one class for each digit.

#Build the model

```
model = tf.keras.models.Sequential()
```

```
model.add(keras.layers.Dense (NB_CLASSES,  
                               input_shape=(RESHAPED,),  
                               name='dense_layer',  
                               activation='softmax'))
```

The final layer is a single neuron with the activation function 'softmax', which is a generalization of the sigmoid function. A softmax "squashes" a K-dimensional vector of arbitrary real values into a K-dimensional vector of real values in the range $(0,1)$, so that they all add up to 1. In our case, it aggregates ten answers provided by the previous layer with ten neurons.

Once we define the model, we have to compile it so that it can be executed by TensorFlow.

Firstly, we need to select an **optimizer**, which is the specific algorithm used to update weights while we train our model. A complete list of optimizers is at

https://www.tensorflow.org/api_docs/python/tf/keras/optimizers.

Second, we need to select an **objective function**, which is used by the optimizer to navigate the space of weights (**frequently objective functions are called either lost functions or cost functions** and the process of optimization is defined as a process of loss minimization).

Third, we need to evaluate the trained model.

Some common choices for objective function (a complete list of loss functions is at https://www.tensorflow.org/api_docs/python/tf/keras/losses) are:

- mse, which defines the mean squared error between the predictions and true values. Note that this objective function is the average of all the mistakes made in each prediction.
- binary_crossentropy, which defines the binary logarithmic loss. Note that this objective function is suitable for binary labels prediction
- categorical_crossentropy, which defines the multiclass logarithmic loss. Categorical cross-entropy compares the distribution of the predictions with the true distribution, with the probability of the true class set to 1 and 0 for the other classes. Note that this objective function is suitable for multi-class label predictions. It is also the default choice with softmax activation. A complete list of loss functions is at https://www.tensorflow.org/api_docs/python/tf/keras/losses.

Some common choices for metrics (a complete list of metrics is at https://www.tensorflow.org/api_docs/python/tf/keras/metrics) are:

- Accuracy, defined as the proportion of correct predictions with respect to the total number of predictions.
- Precision, defined as the proportion of correct positive predictions with respect to the number of correct and incorrect positive predictions
- Recall, defined as the proportion of correct positive predictions with respect to the actual number of positive predictions

Metrics are similar to objective functions, with the only difference being that they are not used for training a model, only for evaluating the model. The loss function is used to optimize your network. This is the function minimized by the selected optimizer. Instead, a metric is used to judge the performance of your network. When compiling a model in TensorFlow, it is possible to select the optimizer, the loss function, and the metric used together with a given model:

#compiling the model.

```
model.compile(optimizer='SDG',  
              loss='categorical_crossentropy',  
              metrics=['accuracy'])
```

Stochastic Gradient Descent (SGD) is a particular kind of optimization algorithm used to reduce the mistakes made by neural networks after each training epoch.

Once the model is compiled, it can then be trained with the `fit()` method, which specifies a few parameters:

- **epochs** is the number of times the model is exposed to the training set. At each iteration the optimizer tries to adjust the weights so that the objective function is minimized.
- **batch_size** is the number of training instances observed before the optimizer performs a weight update; there are usually many batches per epoch.

#Training the model

```
model.fit(X_train, Y_train,  
         batch_size=BATCH_SIZE, epochs=EPOCHS,  
         verbose=VERBOSE, validation_split=VALIDATION_SPLIT)
```

#evaluate the model

```
test_loss, test_acc = model.evaluate(X_test, Y_test)  
print('Test accuracy:', test_acc)
```

Once the model is trained, we can evaluate it on the test set that contains new examples never seen by the model during the training phase

Running a simple TensorFlow net and establishing a baseline

Full code: https://github.com/PacktPublishing/Deep-Learning-with-TensorFlow-and-Keras-3rd-edition/blob/main/Chapter_1/mnist.py

So, let's see what happens when we run the code:

```
Model: "sequential"

Layer (type)                 Output Shape              Param #
-----
dense_layer (Dense)          (None, 10)                7850
-----

Total params: 7,850
Trainable params: 7,850
Non-trainable params: 0
-----

Train on 48000 samples, validate on 12000 samples
Epoch 1/200
48000/48000 [=====] - 1s 31us/sample - loss: 2.1276 -
accuracy: 0.2322 - val_loss: 1.9508 - val_accuracy: 0.3908
Epoch 2/200
48000/48000 [=====] - 1s 23us/sample - loss: 1.8251 -
accuracy: 0.5141 - val_loss: 1.6848 - val_accuracy: 0.6277
Epoch 3/200
48000/48000 [=====] - 1s 25us/sample - loss: 1.5992 -
accuracy: 0.6531 - val_loss: 1.4838 - val_accuracy: 0.7150
Epoch 4/200
48000/48000 [=====] - 1s 27us/sample - loss: 1.4281 -
accuracy: 0.7115 - val_loss: 1.3304 - val_accuracy: 0.7551
Epoch 5/200

Epoch 199/200
48000/48000 [=====] - 1s 22us/sample - loss: 0.3684 -
accuracy: 0.8995 - val_loss: 0.3464 - val_accuracy: 0.9071
Epoch 200/200
48000/48000 [=====] - 1s 23us/sample - loss: 0.3680 -
accuracy: 0.8996 - val_loss: 0.3461 - val_accuracy: 0.9070
10000/10000 [=====] - 1s 54us/sample - loss: 0.3465 -
accuracy: 0.9071
Test accuracy: 0.9071
```

First the net architecture is dumped and we can see the different types of layers used, their output shape, how many parameters (i.e., how many weights) they need to optimize, and how they are connected.

Then, the network is trained on 48K samples, and 12K are reserved for validation. Once the neural model is built, it is then tested on 10K samples. We can see that the program runs for 200 iterations and each time accuracy improves. When the training ends, we test our model on the test set and we achieve about 89.96% accuracy on the training dataset, 90.70% on validation and 90.71% on test. This means that nearly 1 in 10 images are incorrectly classified.

Improving the simple net in TensorFlow with hidden layers.

Full code: https://github.com/PacktPublishing/Deep-Learning-with-TensorFlow-and-Keras-3rd-edition/blob/main/Chapter_1/mnist_V2.py

From the above results, we have a baseline of accuracy of 89.96% on the training dataset, 90.70% on validation, and 90.71% on test. It is good starting point, but we can improve it.

An initial improvement is to add additional layers to our network because these additional neurons might intuitively help to learn more complex patterns in the training data.

After the input layer, we have a first dense layer with N_HIDDEN neurons and an activation function 'relu'. This additional layer is considered hidden because it is not directly connected either with the input or with the output.

After the first hidden layer, we have a second hidden layer, again with N_HIDDEN neurons, followed by an output layer with ten neurons, each one of which will fire when the relative digit is recognized.

The following code defines this new network:

```
import tensorflow as tf

from tensorflow import keras

#network and training parameters.

EPOCHS = 50

BATCH_SIZE = 128

VERBOSE = 1

NB_CLASSES = 10 # number of outputs = number of digits

N_HIDDEN = 128

VALIDATION_SPLIT = 0.2 # how much train is reserved for validation


#Loading MNIST dataset

#verify
```

#you can verify that the split between train and test is 60,000 and 10,000 respectively.

#labels have one-hot representation is automatically applied

mnist = keras.datasets.mnist

(X_train, Y_train), (X_test, Y_test) = mnist.load_data()

#X_train is 6000 rows of 28x28 values; we → reshape it to 6000 x784

RESHAPE = 784

#

X_train = X_train.reshape(60000, RESHAPE)

X_test = X_test.reshape(10000, RESHAPE)

X_train = X_train.astype('float32')

X_test = X_test.astype('float32')

#normalize inputs to be within in [0,1].

X_train, X_test = X_train / 255.0, X_test / 255.0

print(X_train.shape[0], 'train samples')

print(X_test.shape[0], 'test samples')

#One-hot representation of the labels

Y_train = tf.keras.utils.to_categorical(Y_train, NB_CLASSES)

Y_test = tf.keras.utils.to_categorical(Y_test, NB_CLASSES)

#Build the model

model = tf.keras.models.Sequential()

model.add(keras.layers.Dense (N_HIDDEN,

input_shape=(RESHAPED,),

name='dense_layer', activation = 'relu'))

model.add(keras.layers.Dense (N_HIDDEN,

name='dense_layer2', activation = 'relu'))

model.add(keras.layers.Dense (NB_CLASSES,

name='dense_layer 3', activation='softmax'))

#Summary of the model.

model.summary()

#Compiling the model.

model.compile(optimizer='SDG',

```

        loss='categorical_crossentropy',
        metrics=['accuracy'])

#Training the model

model.fit(X_train, Y_train,

        batch_size=BATCH_SIZE, epochs=EPOCHS,

        verbose=VERBOSE, validation_split=VALIDATION_SPLIT)

```

```

#evaluate the model

test_loss, test_acc = model.evaluate(X_test, Y_test)

print('Test accuracy:', test_acc)

```

Note that **to_categorical(Y_train, NB_CLASSES)** converts the array Y_train into a matrix with as many columns as there are classes. The number of rows stay the same.

Example:

```

> labels
array([0, 2, 1, 2, 0])

```

then:

```

to_categorical(labels)
array([[ 1.,  0.,  0.],
       [ 0.,  0.,  1.],
       [ 0.,  1.,  0.],
       [ 0.,  0.,  1.],
       [ 1.,  0.,  0.]], dtype=float32)

```

Let's run the code and see what results we get with this multi-layer network:

```

Layer (type)                Output Shape              Param #
=====
dense_layer (Dense)          (None, 128)               100480
dense_layer_2 (Dense)        (None, 128)               16512
dense_layer_3 (Dense)        (None, 10)                1290
=====
Total params: 118,282
Trainable params: 118,282
Non-trainable params: 0
=====
Train on 48000 samples, validate on 12000 samples
Epoch 1/50
48000/48000 [=====] - 3s 63us/sample - loss: 2.2507
accuracy: 0.2086 - val_loss: 2.1592 - val_accuracy: 0.3266

```

```
Epoch 49/50
48000/48000 [=====] - 1s 30us/sample - loss: 0.3347 -
accuracy: 0.9075 - val_loss: 0.3126 - val_accuracy: 0.9136
Epoch 50/50
48000/48000 [=====] - 1s 28us/sample - loss: 0.3326 -
accuracy: 0.9081 - val_loss: 0.3107 - val_accuracy: 0.9140
10000/10000 [=====] - 0s 40us/sample - loss: 0.3164 -
accuracy: 0.9118
Test accuracy: 0.9118
```

By adding two hidden layers we reached 90.81% on the training dataset, 91.40% on the validation, and 91.18% on the test. This means that we have increased accuracy on the test dataset with respect to the previous network, and we have reduced the number of iterations from 200 to 50.

If you want, you can play by yourself and see what happens if you add only one hidden layer instead of two or if you add more than two layers.

Note that improvement stops (or it become almost imperceptible) after a certain number of epochs. In machine learning this is a phenomenon called convergence.

Further improving the simple net in TensorFlow with dropout

Full code: https://github.com/PacktPublishing/Deep-Learning-with-TensorFlow-and-Keras-3rd-edition/blob/main/Chapter_1/mnist_V3.py

In this scenario, we decide to randomly drop – with DROPOUT probability. Some of the values propagated inside our internal dense network of hidden layers during training (regularization). This idea randomly dropping a few values can improve our performance. **The idea behind this improvement is that random dropouts force the network to learn redundant patterns that are useful for better generalization:**

```
import tensorflow as tf
import numpy as np
from tensorflow import keras

#network and training parameters.
EPOCHS = 200
BATCH_SIZE = 128
VERBOSE = 1
NB_CLASSES = 10 # number of outputs = number of digits
N_HIDDEN = 128
VALIDATION_SPLIT = 0.2 # how much train is reserved for validation
```

DROPOUT = 0.3

#Loading MNIST dataset

#labels have one-hot representation is automatically applied

mnist = keras.datasets.mnist

(X_train, Y_train), (X_test, Y_test) = mnist.load_data()

#X_train is 6000 rows of 28x28 values; we → reshape it to 6000 x784

RESHAPE = 784

#

X_train = X_train.reshape(60000, RESHAPE)

X_test = X_test.reshape(10000, RESHAPE)

X_train = X_train.astype('float32')

X_test = X_test.astype('float32')

#normalize inputs to be within in [0,1].

X_train, X_test = X_train / 255.0, X_test / 255.0

print(X_train.shape[0], 'train samples')

print(X_test.shape[0], 'test samples')

#One-hot representation of the labels

Y_train = tf.keras.utils.to_categorical(Y_train, NB_CLASSES)

Y_test = tf.keras.utils.to_categorical(Y_test, NB_CLASSES)

#Build the model

model = tf.keras.models.Sequential()

model.add(keras.layers.Dense (N_HIDDEN,

input_shape=(RESHAPED,),

name='dense_layer', activation = 'relu'))

model.add(keras.layers.Dropout (DROPOUT))

model.add(keras.layers.Dense (N_HIDDEN,

name='dense_layer2', activation = 'relu'))

model.add(keras.layers.Dropout (DROPOUT))

model.add(keras.layers.Dense (NB_CLASSES,

name='dense_layer 3', activation='softmax'))

```

#Summary of the model.

model.summary()

#compiling the model.

model.compile(optimizer='SDG',

              loss='categorical_crossentropy',

              metrics=['accuracy'])

#Training the model

model.fit(X_train, Y_train,

        batch_size=BATCH_SIZE, epochs=EPOCHS,

        verbose=VERBOSE, validation_split=VALIDATION_SPLIT)

#evaluate the model

test_loss, test_acc = model.evaluate(X_test, Y_test)

print('Test accuracy:', test_acc)

```

Let's run the code for 200 iterations as before and we see that this net achieves an accuracy of 91.70% on training, 94.42% on validation, and 94.15% on testing:

```

Epoch 199/200
48000/48000 [=====] - 2s 45us/sample - loss: 0.2850 -
accuracy: 0.9177 - val_loss: 0.1922 - val_accuracy: 0.9442
Epoch 200/200
48000/48000 [=====] - 2s 42us/sample - loss: 0.2845 -
accuracy: 0.9170 - val_loss: 0.1917 - val_accuracy: 0.9442
10000/10000 [=====] - 1s 61us/sample - loss: 0.1927 -
accuracy: 0.9415
Test accuracy: 0.9415

```

Note that it has been frequently observed that networks with random dropouts in internal hidden layers can “generalize” better on unseen examples contained in test sets. During testing, there is no dropout. In shorts, it is generally, a good approach to test how a net performs when some dropout function is adopted.

Besides that, note that training accuracy should still be above test accuracy; otherwise, we might be not training for long enough. This is the case above result, therefore, we should increase the number of epochs.

Testing different optimizer in TensorFlow

Full code: https://github.com/PacktPublishing/Deep-Learning-with-TensorFlow-and-Keras-3rd-edition/blob/main/Chapter_1/mnist_V4.py

It is useful to start developing some intuition about how networks are trained. One popular technique known as gradient descent (GD). Imagine a generic cost function $C(w)$ on one single variable w like in figure 1.12:

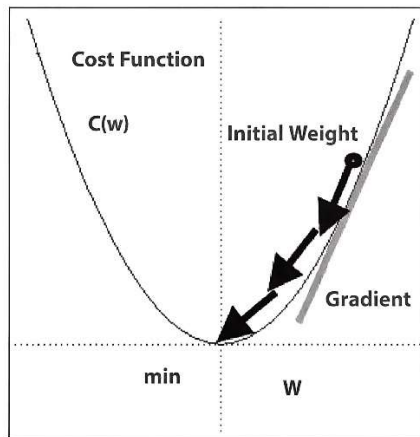


Figure 1.12: An example of GD optimization

GD can be seen as hiker who needs to navigate down a steep slope and aims to enter a ditch. The slope represents the function C while the ditch represents the minimum $C_{\min}$. The hiker has a starting point w_0 . The hiker moves little by little; imagine there is almost zero visibility, so the hiker cannot see where to go automatically, and they proceed in a zigzag. At each step r , the gradient is the direction of maximum increase.

At each step the hiker can decide how big a stride to take before the next stop. This is the so called “learning rate” $\eta \geq 0$ in GD jargon. Note that if η is too small, then the hiker will move slowly, however if η is too high, then the hiker will possibly miss the ditch by stepping over it.

TensorFlow implements a fast variant of GD known as Stochastic Gradient Descent (SDG) and many more advanced optimization techniques such as RMSProp and Adam. RMSProp and Adam include the concept of momentum (a velocity component). In addition to the acceleration component that SGD has. This allows faster convergence at the cost of more computation. Think about a hiker who starts to move in one direction and then decides to change direction but remembers previous choices. It can be proven that momentum helps accelerate SGD in the relevant direction and dampens oscillations.

SGD was our default choice so far. So now let's try the other two. It is very simple; we just need to change a few lines:

```
#compiling the model.
```

```
model.compile(optimizer='RMSProp',  
              loss='categorical_crossentropy', metrics=['accuracy'])
```

Layer (type)	Output Shape	Param #
dense_layer (Dense)	(None, 128)	100480
dropout_2 (Dropout)	(None, 128)	0
dense_layer_2 (Dense)	(None, 128)	16512
dropout_3 (Dropout)	(None, 128)	0
dense_layer_3 (Dense)	(None, 10)	1290

=====
 Total params: 118,282
 Trainable params: 118,282
 Non-trainable params: 0
 =====

Train on 48000 samples, validate on 12000 samples
 Epoch 1/10
 48000/48000 [=====] - 2s 48us/sample - loss: 0.4715 - accuracy: 0.8575 - val_loss: 0.1820 - val_accuracy: 0.9471

Epoch 2/10
 48000/48000 [=====] - 2s 36us/sample - loss: 0.2215 - accuracy: 0.9341 - val_loss: 0.1268 - val_accuracy: 0.9361
 Epoch 3/10
 48000/48000 [=====] - 2s 39us/sample - loss: 0.1684 - accuracy: 0.9497 - val_loss: 0.1198 - val_accuracy: 0.9651
 Epoch 4/10
 48000/48000 [=====] - 2s 43us/sample - loss: 0.1459 - accuracy: 0.9569 - val_loss: 0.1059 - val_accuracy: 0.9710
 Epoch 5/10
 48000/48000 [=====] - 2s 39us/sample - loss: 0.1273 - accuracy: 0.9623 - val_loss: 0.1059 - val_accuracy: 0.9696
 Epoch 6/10
 48000/48000 [=====] - 2s 36us/sample - loss: 0.1177 - accuracy: 0.9659 - val_loss: 0.0941 - val_accuracy: 0.9731
 Epoch 7/10
 48000/48000 [=====] - 2s 35us/sample - loss: 0.1083 - accuracy: 0.9671 - val_loss: 0.1009 - val_accuracy: 0.9715
 Epoch 8/10
 48000/48000 [=====] - 2s 35us/sample - loss: 0.0971 - accuracy: 0.9706 - val_loss: 0.0950 - val_accuracy: 0.9758
 Epoch 9/10
 48000/48000 [=====] - 2s 35us/sample - loss: 0.0969 - accuracy: 0.9718 - val_loss: 0.0985 - val_accuracy: 0.9745
 Epoch 10/10
 48000/48000 [=====] - 2s 35us/sample - loss: 0.0873 - accuracy: 0.9743 - val_loss: 0.0966 - val_accuracy: 0.9762
 10000/10000 [=====] - 1s 2ms/sample - loss: 0.0922 - accuracy: 0.9764
 Test accuracy: 0.9764

As you can see, **RMSProp** is faster than **SDG** since we are able to achieve in only **10 epochs** an accuracy of **97.43%** on the training dataset, **97.62%** on validation, and **97.64%** on test. That's significant improvement on SDG. Now that we have a very fast optimizer, let us try to increase significantly the number of epochs up to 250, and we get 98.99% on the training dataset, 97.66% on validation, and 97.77% on test:

```

Epoch 248/250
48000/48000 [=====] - 2s 40us/sample - loss: 0.0506 -
accuracy: 0.9904 - val_loss: 0.3465 - val_accuracy: 0.9762
Epoch 249/250
48000/48000 [=====] - 2s 40us/sample - loss: 0.0490 -
accuracy: 0.9905 - val_loss: 0.3645 - val_accuracy: 0.9765
Epoch 250/250
48000/48000 [=====] - 2s 39us/sample - loss: 0.0547 -
accuracy: 0.9899 - val_loss: 0.3353 - val_accuracy: 0.9766
10000/10000 [=====] - 1s 58us/sample - loss: 0.3184 -
accuracy: 0.9779
Test accuracy: 0.9779

```

It is useful to observe how accuracy increases on training and test sets when the number of epochs increase (see figure 1.13). as you can see, these two curves touch at about 15 epochs and therefore there is no need to train further after that point:

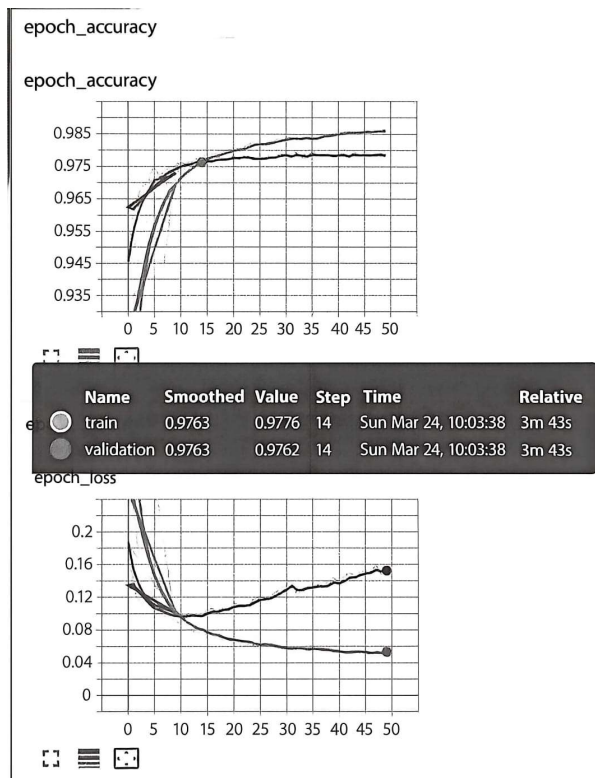


Figure 1.13: An example of accuracy and loss with RMSProp

Let's try the other optimizer, Adam():

```
#compiling the model.
```

```
model.compile(optimizer='Adam',
              loss='categorical_crossentropy', metrics=['accuracy'])
```

As we see, Adam() is slightly better. With Adam we achieve 98.94% accuracy on the training dataset, 97.89% on validation, and 97.82% on test with 50 iterations:

```
Epoch 49/50
48000/48000 [=====] - 3s 55us/sample - loss: 0.0313 -
accuracy: 0.9894 - val_loss: 0.0868 - val_accuracy: 0.9808
Epoch 50/50
48000/48000 [=====] - 2s 51s/sample - loss: 0.0321 -
accuracy: 0.9894 - val_loss: 0.0983 - val_accuracy: 0.9789
10000/10000 [=====] - 1s 66us/step - loss: 0.0964 -
accuracy: 0.9782
Test accuracy: 0.9782
```

Let's plot how accuracy increases on training and test sets when the number of epochs increase (see figure 1.14). You'll notice that by choosing Adam as an optimizer we are able to stop after just about 12 epochs or steps:

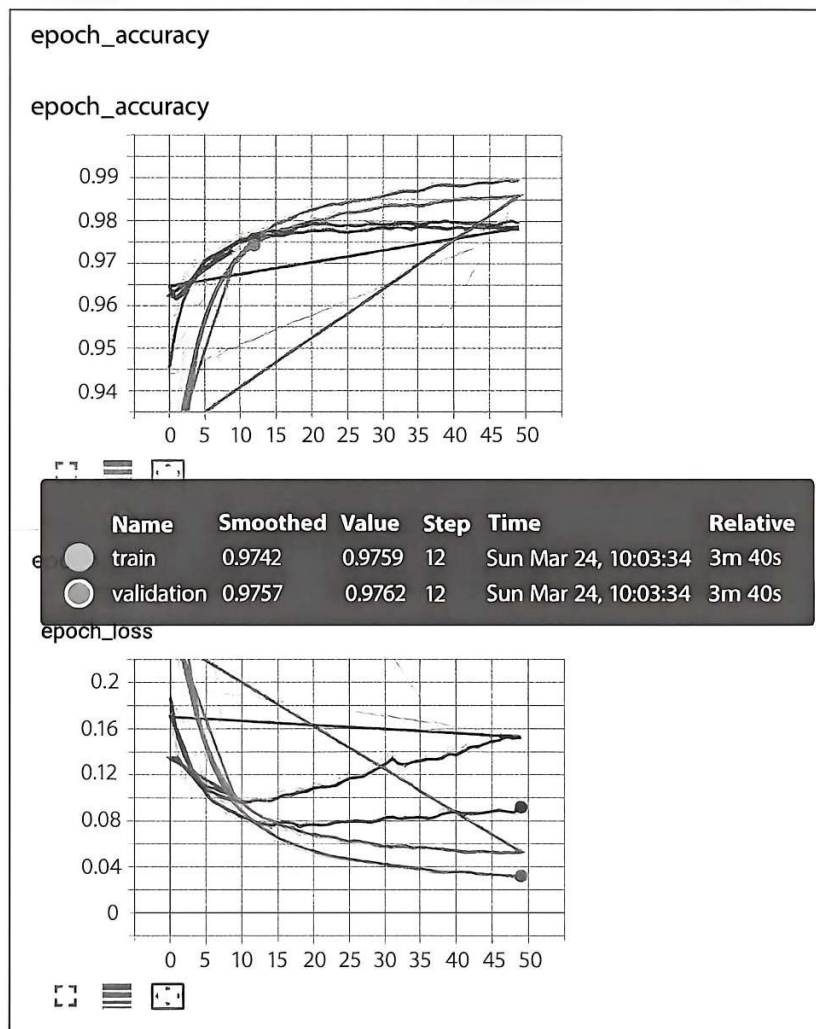


Figure 1.14: An example of accuracy and loss with Adam

Note that this is our fifth variant and remember that our initial baseline was at 90.71% on the test dataset. So far, we've made progressive improvements. However, gains are now more and more difficult to obtain. Note that we are optimizing with a dropout of 30%. For the sake of completeness, it could be useful to report the accuracy of the test dataset for different dropout values (see figure 1.15). In this example, we selected Adam as the optimizer. Note that the choice of optimizer isn't a rule of thumb and we can get different performance depending on the problem-optimizer combination:

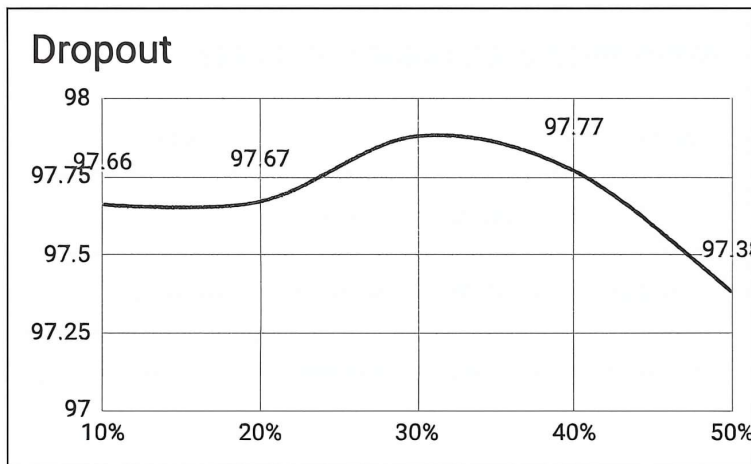


Figure 1.15: An example of changes in accuracy for different dropout values

Increasing the number of epochs

Case: increase the number of epochs used for training e.g. from 20 to 200 will increase the computation time 10 fold, yet gives no gain. The experiment is unsuccessful, but we have learned that if we spend more time learning, we will not necessarily improve the result. Learning is more about adopting smart techniques and not necessarily about the time spent in computations.

Let's keep track of our five variants in the following graph:

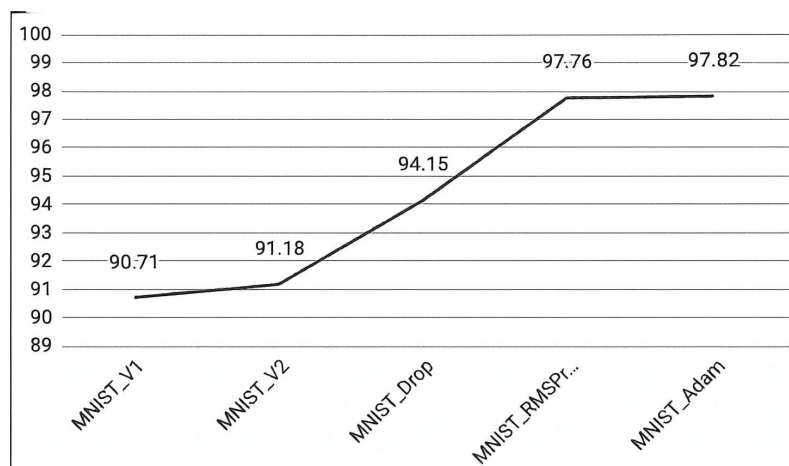


Figure 1.16: Accuracy for different models and optimizers

Controlling the optimizer learning rate

There is another approach we can take that involves changing the learning parameter for our optimizer.

As you can see in figure 1.17, the best value reached by our three experiments [$lr=0.1$, $lr=0.01$, and $lr=0.001$] is 0.1, which is the default learning rate for the optimizer. Good! Adam works well out of the box:

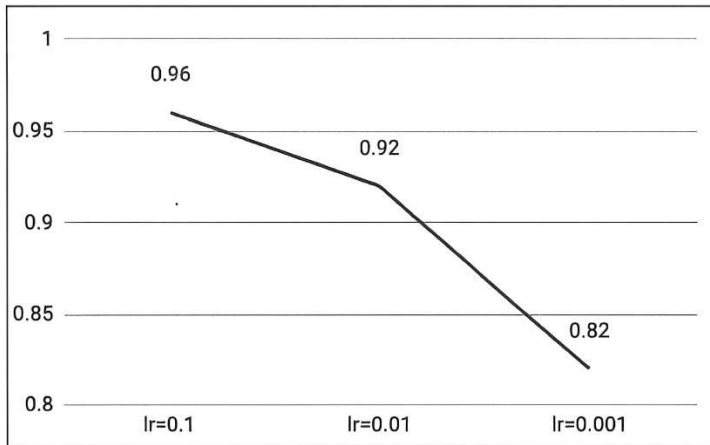


Figure 1.17: Accuracy for different learning rates

Increase the number of internal hidden neurons

Yet another approach involves changing the number of internal hidden neurons. We report the results of the experiments with an increasing number of hidden neurons. We see that by increasing the complexity of the model, the run time increases significantly because there are more and more parameter optimized. However, the gains that we are getting by increasing the size of the network decrease more and more as the network grows (see figure 1.18, figure 1.19, and figure 1.20)

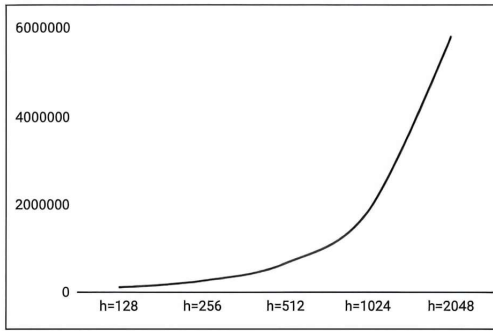


Figure 1.18: Number of parameters for the increasing values of internal hidden neurons

On the other hand, the time needed increases as the size of the internal network increases (see Figure 1.19):

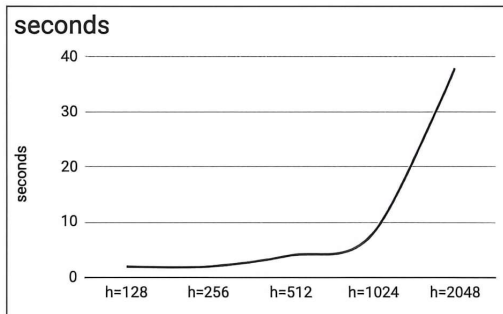


Figure 1.19: Seconds of computation time for the increasing values of internal hidden neurons

Note that increasing the number of hidden neurons after a certain value can reduce the accuracy because the network might not be able to generalize well (as shown in Figure 1.20):

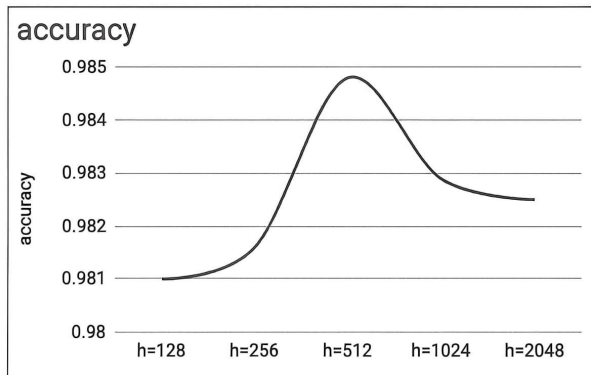


Figure 1.20: Test accuracy for the increasing values of internal hidden neurons

Increase the size of batch computation

GD tries to minimize the cost function on all the examples provided in the training sets and, at the same time, for all the features provided as input. SGD is a much less expensive variant that considers only `BATCH_SIZE` examples. So, let us see how it behaves when we change this parameter. As you can see, the best accuracy value is reached for a `BATCH_SIZE=64` in our four experiments (see figure 1.21):

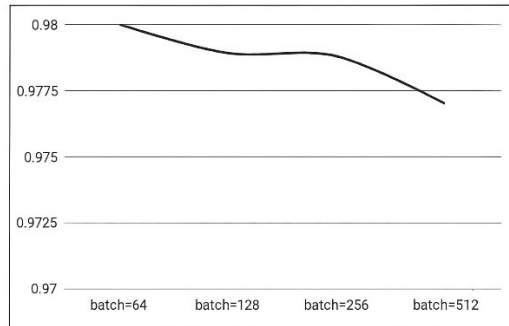


Figure 1.21: Test accuracy for different batch values

Summarizing experiments run to recognizing handwritten digits

	Accuracy		
Model	Training	Validation	Test
Simple	89.96%	90.70%	90.71%
Two hidden layers (128)	90.81%	91.40%	91.18%
Dropout (30%)	91.70%	94.42%	94.15% (200 epochs)
RMSProp	97.43%	97.62%	97.64% (10 epochs)
Adam	98.94%	97.89%	97.82% (10 epochs)

Table 1.1: Summary of experiments with various levels of accuracy

Regularization

Adopting regularization to avoid “overfitting”

There are three different types of regularization used in machine learning:

- L1 regularization (also known as LASSO). The complexity of the model is expressed as the sum of the absolute values of the weights.
- L2 regularization (also known as Ridge). The complexity of the model is expressed as the sum of the squares of the weights.
- ElasticNet regularization. The complexity of the model is captured by a combination of the two techniques above.

Also note that TensorFlow supports L1, L2, and ElasticNet regularization. A complete list of regularizers is at http://www.tensorflow.org/api_docs/python/tf/keras/regularizers.

```

from tf.keras.regularizers import l2, activity_l2
model.add(Dense(64, input_dim=64, W_regularizer=l2(0.01),
               activity_regularizer=activity_l2(0.01)))

```

Understanding Batch Normalization

Batch normalization is another form of regularization and one of the most effective improvements proposed during the last few years. Batch normalization enables us to accelerate training, in some cases by halving the training epochs, and it offers some regularization.

During training, the weights in early layers naturally change and therefore the inputs of later layers can significantly change. In other words, each layer must continuously re-adjust its weight to the different distribution for every batch. This may slow down the model's training greatly.

Sentiment Analysis

The IMDB dataset contains the text of 50,000 movie reviews from the internet movie database. Each review is either positive or negative (for example, thumbs up or thumbs down). The data is split into 25,000 reviews for training and 25,000 reviews for testing. Our goal is that to build a classifier that can predict the binary judgement given the text.

We can easily load IMDB via tf.keras and the sequences of the words in the reviews have been converted to sequences of integers, where each integer represents a specific word in a dictionary. We also have a convenient way of padding sentences to `max_len`, so that we can use all sentences, whether short or long, **as inputs to a neural network with an input vector of fixed size:**

```

import tensorflow as tf

from tensorflow.keras import datasets, layers, models, preprocessing

import tensorflow_datasets as tfds

max_len = 200

n_words = 10000

dim_embedding = 256

EPOCHS = 20

BATCH_SIZE = 500

def load_data():
    # load data.
    (X_train, y_train), (X_test, y_test) = datasets.imdb.load_data(num_words = n_words)

    # Pad sequences with max_len
    X_train = preprocessing.sequence.pad_sequences(X_train, maxlen = max_len)
    X_test = preprocessing.sequence.pad_sequences(X_test, maxlen = max_len)
    return (X_train, y_train), (X_test, y_test)

```

Use a few layers to build a model. **Embedding () layer will map the sparse space of words contained in the reviews into a denser space. This will make computation easier.** In addition, we will use GlobalMaxPooling1D() layer which takes the maximum value of either feature vector from each of the the n_words features. In addition, we have two Dense() layers. The last one is made up of a single neuron with a sigmoid function for making the final binary estimation:

```
def build_model():  
    model = models.Sequential()  
  
    # Input: - embedding layer.  
  
    #The model will take as input an integer matrix of size (batch, input_length).  
  
    #the model will output dimension (input_length, dim_embedding).  
  
    #the largest integer in the input should be no larger  
    #than n_words (vocabulary size).  
  
    model.add(layers.Embedding(n_words,  
                               dim_embedding, input_length = max_len))  
  
    model.add(layers.Dropout(0.3))  
  
  
    #takes the maximum value of either feature vector from each of the n_words features.  
    model.add(layers.GlobalmaxPooling1D())  
  
    model.add(layers.Dense(128, activation='relu'))  
  
    model.add(layers.Dropout(0.5))  
  
    model.add(layers.Dense(1, activation='sigmoid'))  
  
    return model
```

now we need to train our model and this piece of code is very similar to what we have done with MNIST. Let's see:

```
(X_train, y_train), (X_test, y_test) = load_data()  
  
model = build_model()  
  
model.summary()  
  
model.compile(optimizer = "adam", loss = "binary_crossentropy", metrics = ["accuracy"]  
)  
  
score = model.fit(X_train, y_train,  
                  epochs = EPOCHS,  
                  batch_size = BATCH_SIZE,  
                  validation_data = (X_test, y_test)  
)
```

```
score = model.evaluate(X_test, y_test, batch_size = BATCH_SIZE)
print("\nTest score:", score [0])
print('Test accuracy:', score [1])
```

Layer (type)	Output Shape	Param #
embedding (Embedding)	(None, 200, 256)	2560000
dropout (Dropout)	(None, 200, 256)	0
global_max_pooling1d (Global	(None, 256)	0
dense (Dense)	(None, 128)	32896
dropout_1 (Dropout)	(None, 128)	0
dense_1 (Dense)	(None, 1)	129
Total params: 2,593,025		
Trainable params: 2,593,025		
Non-trainable params: 0		

As shown in the following output, we reach accuracy of 85%, which is not bad at all for a simple network

```
Epoch 20/20
25000/25000 [=====] - 23s 925ms/sample - loss: 0.0053
- accuracy: 0.9991 - val_loss: 0.4993 - val_accuracy: 0.8503
25000/25000 [=====] - 2s 74us/sample - loss: 0.4993 -
accuracy: 0.88503

Test score: 0.4992710727453232
Test accuracy: 0.85028
```

Layer embedding:

```
model.add
(layers.Embedding(n_words,
dim_embedding, input_length =
max_len))
```

Potongan kode tersebut adalah langkah pertama dalam membangun jaringan saraf untuk data teks (seperti klasifikasi sentimen atau chatbot). Lapisan **Embedding** berfungsi untuk mengubah angka indeks kata menjadi vektor padat (*dense vector*) yang memiliki makna semantik.

- `n_words` : Ukuran kosakata (*vocabulary size*). Jumlah total kata unik yang diketahui oleh model Anda.
- `dim_embedding` : Dimensi vektor. Setiap kata akan diwakili oleh daftar angka (vektor) sebanyak dimensi ini (misalnya 100 atau 300).
- `input_length` : Panjang kalimat. Karena jaringan saraf memerlukan input dengan ukuran tetap, kita menentukan berapa banyak kata dalam satu baris data (misalnya 50 kata).

Sebelum masuk ke layer ini, teks Anda biasanya diubah menjadi angka (Tokenisasi).

Contoh: "Saya belajar AI" → [12, 45, 89]

Tanpa Embedding, angka 12 dan 45 hanyalah identitas. Namun, dengan **Embedding Layer**, model akan belajar:

- Kata yang memiliki makna serupa (misalnya "Kucing" dan "Anjing") akan memiliki posisi vektor yang berdekatan di ruang matematis.
- Kata yang tidak berhubungan akan saling menjauh.

Jika Anda memiliki 10.000 kata unik:

- **One-Hot Encoding**: Setiap kata membutuhkan 10.000 kolom (sangat boros memori dan tidak punya makna hubungan antar kata).
- **Embedding**: Setiap kata hanya membutuhkan, misalnya, 128 kolom. Ini jauh lebih efisien secara komputasi dan mampu menangkap "konteks".

Misalkan `dim_embedding = 3`. Maka kata "AI" mungkin akan terlihat seperti ini di dalam memori komputer:

"AI" -> [0.12, -0.59, 0.88]

Angka-angka ini awalnya acak, tetapi **berubah (terupdate)** selama proses training, sama seperti *weights* pada MLP yang kita bahas sebelumnya.

Hyperparameter tuning and AutoML

The experiments defined above give some opportunities for fine-tuning net. **For a given neural network, there are indeed multiple parameters that can be optimized (such as the number of hidden neurons, batch size, number of epochs, and many more according to the complexity of the net itself).** These parameters are called "hyperparameters" to distinguish them from parameters of the network itself, i.e. the values of the weights and biases.

Hyperparameter tuning is the process of finding the optimal combination of those hyperparameters that minimize cost functions. **AutoML is a set of research techniques aimed at both automatically tuning hyperparameters and searching automatically for optimal network architecture.**

Predicting Output

Once a net is trained, it can of course be used for making predictions. In TensorFlow, this is very simple. We can use this method:

```
#Making predictions.
```

```
Predictions = model.predict(X)
```

For a given input, several types of output can be computed including a method `model.evaluate()` used to compute the loss values, a method `model.predict_classes()` used to compute category outputs, and a method `model.predict_proba()` used to compute class probabilities.

A practical overview of backpropagation

Multi-layer perceptron learns from training data through a process called backpropagation. **The process can be described as a way of progressively correcting mistakes as soon as they are detected.** Each neural network layer has an associated set of weights that determine the output values for a given set of inputs. Neural network can have multiple hidden layers.

At the beginning, all the weights have some random assignment. Then the neural network is activated for each input in the training set: values are propagated forward from the input stage through the hidden stages to the output stage where a prediction is made. Since we know the true observed value in the training set, it is possible to calculate the error made in the prediction. The key intuition for backtracking is to propagate the error back using an appropriate optimizer algorithm such as a GD to adjust the neural network weights with the goal of reducing the error. The process of forward propagation from input to output and backward propagation of errors is repeated several times until the error gets below a predefined threshold.

The whole process is represented in figure 1.29 below:

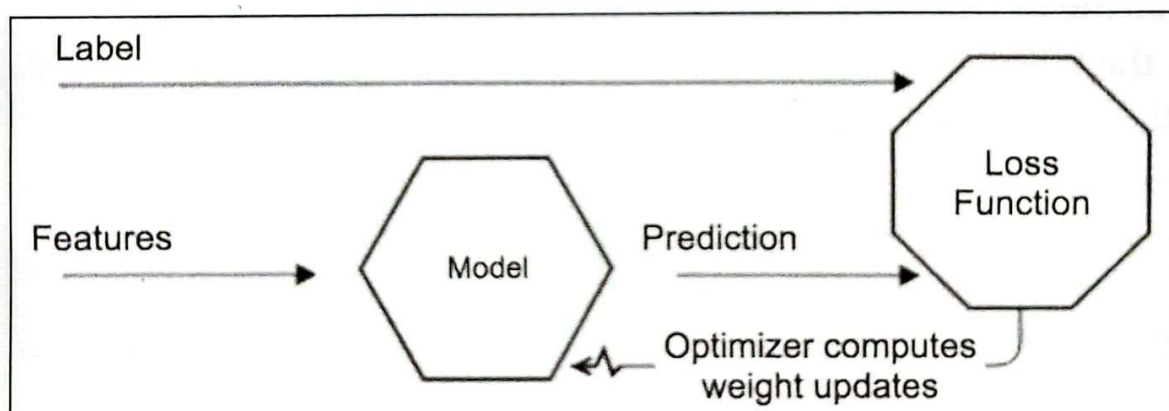


Figure 1.29: Forward propagation and backward propagation